REJECTIONGYM

Production-Ready Full-Stack Cloud Application
Technical Documentation

BTS Cloud Computing
Lycée Guillaume Kroll, Luxembourg
Cohort: BCLC1-25

26 June 2026

Table of Contents

Right-click and select Update Field to refresh page numbers.

1. Project Overview

RejectionGym is a full-stack cloud web application designed to help users build confidence through daily challenges. The application combines social skills challenges, career growth tasks, fitness goals, and comfort zone pushes into a gamified experience with streak tracking, statistics, and shareable progress reports.

The project was built as a BTS Cloud Computing school project, demonstrating professional-grade architecture including Docker containerization, CI/CD pipelines, reverse proxy with HTTPS, automated backups, comprehensive testing, and clean code structure. The application follows a strict 3-tier architecture: Frontend (React SPA) → Backend API (Express) → Database (PostgreSQL).

The name RejectionGym reflects the core philosophy: deliberately seeking rejection and discomfort to build resilience and confidence. Users complete daily challenges, upload proof of completion, track their streaks, and generate shareable progress reports to celebrate their growth.

Key Features

- Daily challenge system with difficulty progression and category-based organization
- Streak tracking with intelligent consecutive-day logic (reset on gaps)
- File upload to MinIO S3-compatible storage with UUID-based filenames and mime-type validation
- Visual statistics with Recharts (streak line chart, completion pie chart, category breakdown)
- Shareable public reports with unique URLs and social media meta tags
- Admin dashboard for user management and platform statistics
- Responsive mobile-first design with Tailwind CSS

Project Scope

This project is designed to be deployed on a Linux VM with Docker Compose, Nginx reverse proxy, and Let's Encrypt SSL certificates. The entire stack is containerized with health checks, automatic restarts, and nightly database backups. The CI/CD pipeline runs on GitHub Actions with lint, test, build, and deploy stages.

2. Architecture & Design

2.1 3-Tier Architecture

The application follows a strict 3-tier separation with no direct database access from the frontend:

- Tier 1 — Presentation Layer: React 18 SPA served via Vite and Nginx
- Tier 2 — Application Layer: Express API with TypeScript, handling all business logic
- Tier 3 — Data Layer: PostgreSQL 16 with Prisma ORM, plus MinIO for object storage

2.2 Monorepo Structure

```
rejectiongym/\n├── apps/\n│   ├── web/                # React frontend (Vite)\n│
└── api/                # Express backend\n├── packages/\n│   ├── shared/
# Zod schemas + shared types\n│   └── database/           # Prisma schema, client,
migrations, seed\n├── docker-compose.yml\n├── docker-compose.prod.yml\n├──
nginx.conf\n├── Makefile\n└── .github/workflows/ci.yml
```

2.3 Network Flow

The network flow is implemented as follows:

```
[User] → [Nginx :443] → /api → [Express API :3000]\n                    → /     →
[React Static :80]\n[Express API] → [PostgreSQL :5432]\n[Express API] → [MinIO
:9000]
```

Nginx acts as the single entry point, handling SSL termination, gzip compression, security headers, and routing. The API and web containers are isolated within a Docker bridge network named rejectiongym. All inter-service communication is internal to the Docker network.

3. Tech Stack

3.1 Frontend Technologies

Technology	Version	Purpose
React	18.3	UI library with hooks and functional components
Vite	5.2	Build tool and dev server
TypeScript	5.4	Type-safe development with strict mode
Tailwind CSS	3.4	Utility-first CSS framework
React Router DOM	6.23	Client-side routing
TanStack Query	5.37	Server state management (caching, refetch)
Recharts	2.12	Data visualization charts
react-hook-form	7.51	Form management with validation
react-hot-toast	2.4	Toast notifications
react-error-boundary	4.0	Error handling at route level

3.2 Backend Technologies

Technology	Version	Purpose
Node.js	20 LTS	Runtime environment
Express	4.19	HTTP server framework
TypeScript	5.4	Type-safe development with strict mode
Prisma	5.14	ORM with schema-first migrations
PostgreSQL	16	Relational database
bcryptjs	2.4	Password hashing (12 rounds)
jsonwebtoken	9.0	JWT signing and verification
@aws-sdk/client-s3	3.577	S3-compatible MinIO client
multer	1.4	Multipart file upload handling
helmet	7.1	Security headers
winston	3.13	Structured logging
swagger-jsdoc	6.2	OpenAPI documentation generation

3.3 Infrastructure

Technology	Version	Purpose
Docker	latest	Containerization platform
Docker Compose	3.9	Multi-service orchestration
Nginx	alpine	Reverse proxy and static file serving
MinIO	latest	S3-compatible object storage
Certbot	latest	Let's Encrypt SSL certificate management
GitHub Actions	—	CI/CD pipeline automation

4. Database Schema

4.1 Entity Model

The database is modeled using Prisma 5 with PostgreSQL 16 as the backend. The schema consists of 6 main models and 3 enums, with proper indexes, unique constraints, and explicit onDelete cascade rules for referential integrity.

Model	Description	Key Constraints
User	Application users with role-based access	email UK, username UK
ChallengePack	Preset challenge collections	—
Challenge	Individual challenges (preset or custom)	FK to pack, user
Completion	User completion records with streaks	FK to challenge, user
File	Uploaded files stored in MinIO	storageKey UK
Report	Shareable progress reports	publicSlug UK

4.2 Prisma Schema

The schema uses explicit onDelete rules to ensure data integrity. When a user is deleted, all their challenges, completions, and reports are cascade-deleted. When a challenge pack is deleted, its challenges are set to null (soft reference) rather than deleted.

```
model User {\n  id String @id @default(uuid())\n  email String @unique\n  username
String @unique\n  passwordHash String\n  role Role @default(USER)\n  createdAt
DateTime @default(now())\n  updatedAt DateTime @updatedAt\n  challenges
Challenge[]\n  completions Completion[]\n  reports Report[]\n\n  @@index([email])\n
@@index([role])\n}
```

Indexes are placed on all foreign key columns and frequently queried fields: User.email, User.role, Challenge.userId, Challenge.scheduledDate, Completion.userId, Completion.challengeId, Report.userId, File.challengeId, and File.completionId. This ensures optimal query performance for the most common access patterns.

Enums

Enum	Values	Used In
Role	USER, ADMIN	User.role
Category	SOCIAL, CAREER, FITNESS, COMFORT_ZONE	ChallengePack, Challenge
Difficulty	EASY, MEDIUM, HARD	ChallengePack, Challenge

5. Backend API

The Express backend exposes a RESTful API under the /api prefix. All routes return a consistent JSON envelope: { success: boolean, data?: T, error?: string, message?: string }. The API is documented via Swagger/OpenAPI 3.0 at /api/docs.

5.1 Authentication

Method	Endpoint	Auth	Description
POST	/api/auth/register	No	Create account (Zod validation, bcrypt 12 rounds)
POST	/api/auth/login	No	Verify password, return JWT in HTTP-only cookie
POST	/api/auth/logout	Yes	Clear JWT cookie
GET	/api/auth/me	Yes	Return current user (id, email, username, role)

The JWT token is stored in an HTTP-only, Secure, SameSite=Strict cookie named token. This prevents XSS attacks from stealing the token while ensuring the browser automatically sends it with every API request. The token expires after 7 days.

5.2 Challenges

Method	Endpoint	Auth	Description
GET	/api/challenges/daily	Yes	Today's scheduled challenge for logged-in user
POST	/api/challenges	Yes	Create a custom user challenge
GET	/api/challenges	Yes	List user challenges with pagination & status filter
GET	/api/challenges/:id	Yes	Challenge detail with files and completion status
PATCH	/api/challenges/:id/complete	Yes	Mark complete + streak calculation
DELETE	/api/challenges/:id	Yes	Soft delete (or hard delete if no completions)

The streak calculation logic is a core business feature: if the last completion was yesterday, the streak increments by 1. If already completed today, the current streak is returned. If the gap between completions is greater than 1 day, the streak resets to 1. This logic is wrapped in a Prisma transaction to ensure atomicity.

5.3 Stats & Reports

Method	Endpoint	Auth	Description

GET	/api/stats	Yes	Current streak, completion rate, category breakdown
POST	/api/reports	Yes	Generate report with unique public slug
GET	/api/reports/:slug	Cond.	Public report if isPublic=true, else owner-only
GET	/api/reports	Yes	List user's reports with pagination
PATCH	/api/reports/:id/visibility	Yes	Toggle public/private (owner only)

5.4 Admin & Middleware

Method	Endpoint	Auth	Description
GET	/api/admin/users	Admin	List all users with pagination (no password hashes)
GET	/api/admin/stats	Admin	Platform-wide statistics

Middleware stack includes: authenticate (JWT cookie verification), authorize (role-based access control), validateRequest (Zod validation with field-level 400 errors), rateLimit (5 requests per 15 min per IP on auth endpoints), errorHandler (centralized production-safe logging), helmet (security headers), and cors (explicit whitelist, no wildcard).

6. Frontend Application

6.1 Pages & Routes

Route	Page	Auth	Description
/	Landing	No	Hero section, value prop, CTA to register
/login	Login	No	Auth form with RHF + Zod validation
/register	Register	No	Account creation with password requirements
/dashboard	Dashboard	Yes	Daily challenge, streak counter, stats grid
/packs	Browse Packs	Yes	Grid of preset challenge packs
/challenges	Challenge List	Yes	Active/Completed tabs, calendar view
/challenge/:id	Challenge Detail	Yes	Description, complete button, file upload
/stats	Statistics	Yes	Recharts visualizations (streak, completion, category)
/reports	Reports	Yes	List reports, generate new, toggle visibility
/report/:slug	Public Report	No	Standalone shareable page with social meta tags
/admin	Admin Dashboard	Admin	Users table, platform stats, create packs

6.2 State & UI

TanStack Query (React Query) v5 is used for all server state management, providing automatic caching, background refetching, and optimistic updates. The QueryClient is configured with a 5-minute stale time and disabled window-focus refetching to reduce unnecessary API calls.

React Context is used only for auth state (login, logout, user object). This separation keeps server state concerns in TanStack Query while auth lifecycle lives in a dedicated context provider.

UI/UX Standards

- Mobile-first responsive design using Tailwind CSS breakpoints (sm, md, lg)
- Loading skeletons for every async data fetch using Tailwind pulse animation
- Toast notifications via react-hot-toast for all mutations (success, error, loading)

- React Error Boundary on route level to catch render crashes gracefully
- Form validation with Zod schema + react-hook-form with inline error messages
- Protected routes redirect unauthenticated users to /login
- Admin routes redirect non-admin users to /dashboard with a toast warning
- Empty states with descriptive icons and text for no data scenarios
- Color palette: Slate/Zinc neutrals, Emerald for success, Amber for streaks/warnings, Rose for errors

Accessibility

- All form inputs have associated <label> elements with htmlFor
- All buttons have clear text labels or aria-label attributes
- Color contrast meets WCAG AA minimum standards (4.5:1 for body text)
- Focus states are visible on all interactive elements using Tailwind ring utilities

7. Docker & Deployment

7.1 Dev & Prod Stacks

The docker-compose.yml defines 5 services for local development: postgres (PostgreSQL 16 with health checks), minio (object storage with API and console), api (Express backend with live reload via tsx watch), web (React frontend with Vite HMR), and nginx (reverse proxy on port 80).

The docker-compose.prod.yml adds production-specific features: restart: unless-stopped on all services, Nginx on port 443 with SSL certificates, no development volumes (immutable images), a Certbot container for automatic Let's Encrypt renewal, and a backup container running nightly pg_dump via cron.

7.2 Nginx & SSL

The Nginx configuration handles SSL termination, reverse proxying, static file serving, and security headers. The production config listens on port 443 with SSL and redirects all HTTP traffic to HTTPS. The development config listens on port 80 and proxies to the appropriate containers.

Security headers include: X-Frame-Options DENY, X-Content-Type-Options nosniff, X-XSS-Protection 1; mode=block, and Referrer-Policy strict-origin-when-cross-origin. Gzip compression is enabled for JS, CSS, JSON, and XML responses.

Both the API and web use multi-stage Dockerfiles. The API builder stage compiles TypeScript and generates the Prisma client, while the runner stage contains only the compiled dist, node_modules, and prisma schema. The web builder stage compiles the React app, and the serve stage uses Nginx Alpine to serve the static files.

8. CI/CD Pipeline

The .github/workflows/ci.yml defines a 4-stage pipeline triggered on push and pull requests to the main branch:

- Stage 1: Lint & Format — ESLint + Prettier check for both apps. Fails on any lint error.
- Stage 2: Test — Backend (Jest + Supertest) and Frontend (Vitest + RTL) with coverage reports. Fails if coverage drops below 70% on core business logic.
- Stage 3: Build — Docker image builds for api and web, docker-compose up --build verification, health check curl.
- Stage 4: Deploy — SSH to production VM, git pull, docker-compose -f docker-compose.prod.yml up -d --build, run migrations.

The pipeline uses Docker layer caching via GitHub Actions cache to speed up builds. Secrets (JWT_SECRET, database credentials, VM SSH key) are stored in GitHub Secrets and never committed to the repository.

Metric	Minimum	Scope
Branches	70%	Auth, streak, report generation, challenge completion
Functions	70%	Auth, streak, report generation, challenge completion
Lines	70%	Auth, streak, report generation, challenge completion
Statements	70%	Auth, streak, report generation, challenge completion

9. Testing Strategy

Backend Tests

- Unit Tests (Jest): Auth service (hashing, JWT), streak calculation logic, report generation logic
- Integration Tests (Supertest): All API endpoints including auth flow, CRUD operations, file upload with mocked MinIO, admin access control
- Mocking: MinIO operations are mocked in tests to avoid external dependencies

Frontend Tests

- Unit Tests (Vitest + React Testing Library): Form validation, component rendering, utility functions
- Integration Tests: Full page rendering with mocked API responses
- Coverage exclusions: UI components, main.tsx, App.tsx, and config files

The backend uses Jest with ts-jest for TypeScript compilation. The jest.config.js enforces 70% coverage thresholds. The frontend uses Vitest with jsdom environment and v8 coverage provider. Both test configurations are integrated into the CI/CD pipeline and run on every pull request.

10. Security

Security is implemented across multiple layers of the application stack. No single mechanism is relied upon; defense in depth is the guiding principle.

Authentication & Authorization

- Passwords: Bcrypt with 12 rounds, never logged or returned in API responses
- JWT: HTTP-only, Secure, SameSite=Strict cookies with 7-day expiry
- Role separation: Middleware checks the role field; admin routes return 403 for non-admins
- Rate limiting: 5 requests per 15 minutes per IP on /auth/login and /auth/register

Input Validation & Injection Prevention

- Zod strict validation on every endpoint for body, params, and query
- No raw SQL — Prisma ORM prevents SQL injection through parameterized queries
- File upload mime-type whitelist with UUID filenames (no executable extensions)
- 5MB file size limit enforced server-side

Network & Infrastructure

- Helmet.js provides security headers: CSP, HSTS, X-Frame-Options, X-Content-Type-Options, X-XSS-Protection
- CORS whitelist explicitly set from environment variable — no wildcard allowed
- Nginx rate limiting via limit_req_zone (optional configuration)
- SSL/TLS via Let's Encrypt with automatic renewal via Certbot
- All secrets stored in environment variables (JWT_SECRET minimum 32 characters)

Secrets Management

The .env.example file at the repository root contains all required environment keys with empty values. The real .env file is gitignored and never committed. In production, environment variables are injected at runtime and never baked into Docker images. GitHub Secrets are used for CI/CD pipeline credentials and deployment keys.

11. Environment & Setup

Quick Start

To run the application locally with Docker:

```
cp .env.example .env\ndocker compose up -d
```

This starts all services in the background. Wait approximately 30 seconds for the API to initialize the database and run migrations. Then access the application at:

- Frontend: http://localhost:5173
- API: http://localhost:3000/api
- API Docs: http://localhost:3000/api/docs
- MinIO Console: http://localhost:9001 (minioadmin / minioadmin)

Environment Variables

Variable	Example Value	Purpose
DATABASE_URL	postgresql://postgres:postgres@localhost:5432/rejectiongym?schema=public	PostgreSQL connection string
JWT_SECRET	your-secret-key-min-32-chars!!!	JWT signing secret (min 32 chars)
MINIO_ENDPOINT	localhost or minio	MinIO host address
MINIO_PORT	9000	MinIO API port
MINIO_ACCESS_KEY	minioadmin	MinIO root user
MINIO_SECRET_KEY	minioadmin	MinIO root password
API_PORT	3000	Express server port
FRONTEND_URL	http://localhost:5173	Allowed CORS origin

VITE_API_URL	http://localhost:3000/api	Frontend API base URL

Default Users

Email	Password	Role	Purpose
admin@rejectiongym.com	Admin123!	Admin	Platform administration
demo@rejectiongym.com	Demo123!	User	Demo and testing

12. Backup Strategy

The backup strategy is implemented at multiple layers to ensure data durability and recoverability.

Database Backups

- Nightly automated pg_dump from a dedicated backup container running in the production stack
- Backup files stored in /backups volume with date-stamped filenames: db_YYYYMMDD.sql
- Backup volume mounted to the host for persistent storage outside Docker
- Manual backup available via make backup command

MinIO Backups

- MinIO data persisted via Docker volume minio_data
- Volume snapshots as primary recovery mechanism
- Optional: mc mirror to secondary bucket for off-site redundancy

Log Management

- Docker log driver set to json-file with max-size: 10m and max-file: 3
- Backend Winston logs include timestamp, level, requestId, userId, route, method, statusCode, and responseTime
- Nginx access and error logs directed to Docker stdout/stderr

13. Conclusion

RejectionGym demonstrates a complete production-ready full-stack cloud application built with modern technologies and best practices. The 3-tier architecture ensures separation of concerns, while Docker containerization and CI/CD pipelines provide reliable deployment and maintenance workflows.

The application covers all essential aspects of a professional web project: user authentication with role-based access, database design with referential integrity, file storage with validation, API documentation with Swagger, responsive frontend with data visualization, and comprehensive security measures. The monitoring, logging, and backup strategies ensure operational reliability in production environments.

This project serves as a strong foundation for BTS Cloud Computing coursework, demonstrating practical application of containerization, reverse proxying, SSL termination, automated testing, and continuous deployment — all core competencies in modern cloud computing infrastructure.

Future enhancements could include: push notifications for daily challenges, real-time streak leaderboards, integration with wearable fitness devices, social sharing features, and machine learning-based challenge difficulty adjustment based on user completion patterns.